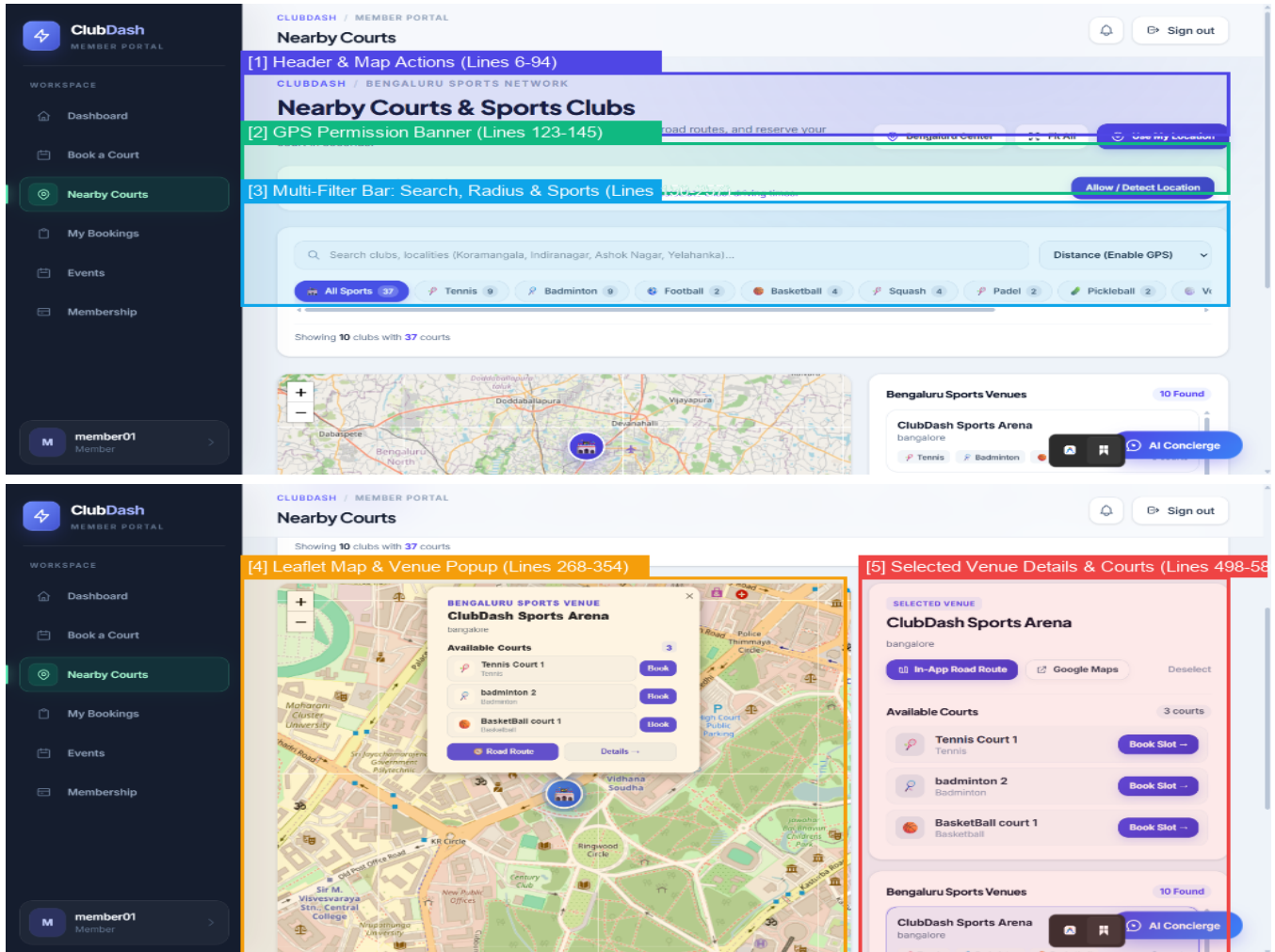


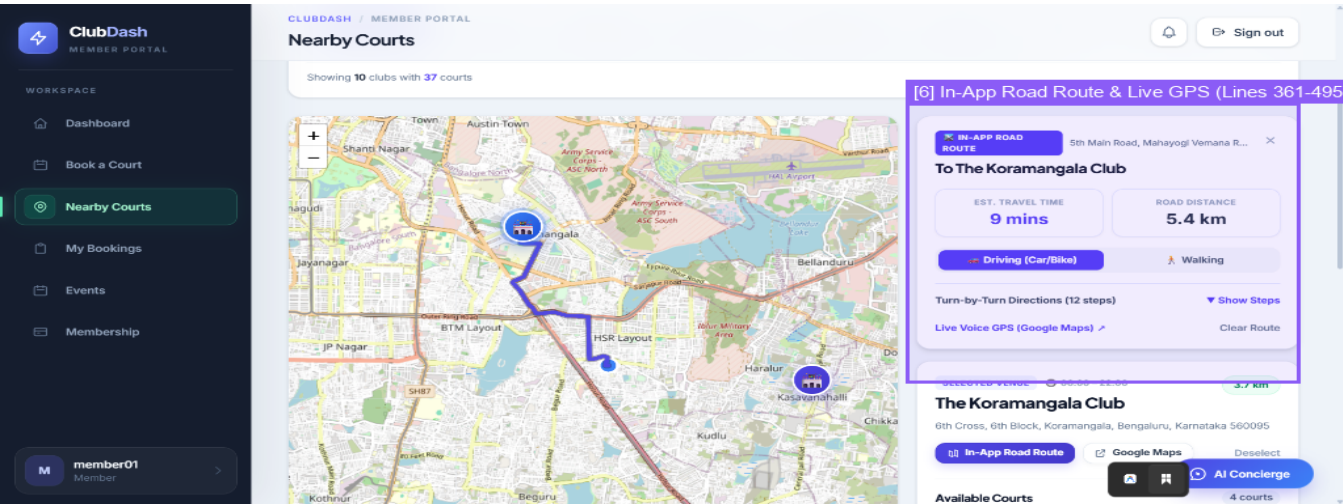
■ ClubDash Master Engineering Specification

Nearby Courts & Sports Clubs (NearbyCourts.vue): Leaflet GIS Map, OSRM Road Routing & Complete Code Breakdown

■ 1. Visual Reference Map: Screenshot to Code Mapping

The live Nearby Courts & GPS Routing system is mapped across 6 functional zones shown in the uploaded screenshots below:





Box #	Nearby Courts UI Section	Template Line Range	Core Responsibilities & State
[1] Indigo	Header & Quick Map Actions	Lines 6 – 94	Center Bengaluru, Fit All venues, and 'Use My Location' GPS button.
[2] Emerald	GPS Permission Banner	Lines 123 – 145	Prompts user to enable browser GPS for accurate driving time & distances.
[3] Sky Blue	Multi-Filter Bar & Sports	Lines 150 – 257	Keyword search, distance radius dropdown, and 11 sport category pill buttons.
[4] Amber	Leaflet Map & Venue Popups	Lines 268 – 354	Interactive OpenStreetMap canvas, venue markers ■■■, and popup court list.
[5] Purple	In-App Road Navigation Card	Lines 361 – 495	Real-time OSRM driving/walking road route with 12 turn-by-turn steps.
[6] Red	Selected Venue Details & Courts	Lines 498 – 580	Operating hours, distance badge, and 'Book Slot →' action buttons.

■ 2. Master Data Origin & API Flow Architecture

NearbyCourts coordinates between PostgreSQL database APIs, browser GPS sensors, and Open Source Routing Machine (OSRM):

UI Variable / Feature	Data Source Type	Exact API Endpoint / Engine	Controller / Service Method	Data Storage / Protocol
clubs	Reactive State (clubs.value)	GET /api/v1/clubs	app/clubs/controllers.py ClubService.get_all_clubs	clubs JOIN courts (PostgreSQL)
userLocation	Browser Sensor (userLocation.value)	navigator.geolocation.getCurrentPosition()	HTML5 Geolocation W3C API	Device GPS / Wi-Fi Triangulation
activeRoute (OSRM)	Routing Engine (fetchRoute())	https://router.project-osrm.org/route/v1/{mode}/{coords}	OSRM Public Routing API	Real OpenStreetMap Road Polyline
Map Tile Imagery	Leaflet TileLayer	https://tile.openstreetmap.org/{z}/{x}/{y}.png	OpenStreetMap Foundation	Standard XYZ Tile Pyramid Cache
Google Maps Directions	External Link	https://www.google.com/maps/dir/?api=1&destination=...	Google Maps Universal URL Scheme	Native Google Maps App / Web

■ 3. Master JavaScript Function & API Binding Traceability Reference

Every JavaScript method in NearbyCourts.vue is mapped to its exact template line number, runtime mechanics, and API endpoint:

Function / Method	Where Used in Template	How & Why It Is Used (Runtime Logic)	API & Data Flow
locateMember()	Line 77: <button @click="locateMember"> Line 141: @click="locateMember(false)"	Requests browser GPS coordinates. Sets userLocation.value = { lat, lng }, centers map, draws blue location marker ■, and calculates distances to all venues.	Invokes: navigator.geolocation.getCurrentPosition()

centerOnBangalore()	Line 30: <button @click="centerOnBangalore">	Centers the Leaflet map to central Bengaluru coordinates [12.9716, 77.5946] at zoom level 12.	Leaflet API: map.setView([12.9716, 77.5946], 12)
fitMap()	Line 59: <button @click="fitMap"> Line 305: @click="fitMap"	Calculates the geographic bounding box of all filtered clubs and animates the map viewport to fit all venue pins.	Leaflet API: map.fitBounds(bounds, { padding: [40, 40] })
toggleFullscreen()	Line 278: <button @click="toggleFullscreen">	Toggles isFullscreen.value boolean. Triggers map.invalidateSize() after CSS resize to ensure smooth tile rendering.	DOM Fullscreen & Leaflet resize event.
selectSport(sport)	Line 215: <button @click="selectSport(sport.value)">	Updates selectedSport.value (e.g. 'tennis', 'badminton'), filters club markers, and re-renders pins on the Leaflet map canvas.	Client-side reactive filter; triggers applyFilters() → drawClubs().
onSearchChange()	Line 174: <input @input="onSearchChange">	Filters venues in real time by club name, locality (Koramangala, Indiranagar, etc.), or court sport type as the user types.	Debounce filter updater.
calculateRoute(club)	Line 541: <button @click="calculateRoute(selectedClub)">	Fetches real turn-by-turn road geometry from user GPS (or city center) to destination venue. Decodes polyline and renders dual-stroke road line on the map.	Calls: fetchRoute(startLat, startLng, destLat, destLng, mode) via OSRM HTTP API
changeRouteMode(mode)	Line 422: @click="changeRouteMode('driving')" Line 434: @click="changeRouteMode('walking')"	Switches navigation mode between Driving (car/bike) and Walking, immediately re-calculating travel time and road steps.	Re-queries OSRM endpoint with mode='driving' or mode='walking'.
clearRoute()	Line 386: @click="clearRoute" Line 490: @click="clearRoute"	Clears the active road navigation polyline layer from Leaflet, closes turn-by-turn drawer, and refits map view.	Leaflet API: routeLayer.clearLayers()
navigateToBook(club, court)	Line 577: <button @click="navigateToBook(selectedClub, court)">	Deep-links the member to the Court Booking page with pre-filled query parameters: /member/book-court?club_id=X&court_id=Y.	Vue Router navigation to BookCourt.vue.
openDirections(club)	Line 482: <button @click="openDirections(club)">	Generates a universal Google Maps directions link with GPS origin and destination coordinates, opening in a new tab.	Opens: https://www.google.com/maps/dir/?api=1&destination=...

■ 4. Complete Template Deep Dive (Lines 1 to 580)

A. [Box 1] Page Header & Quick Map Actions (Lines 6 – 94)

```

6: <div class="flex flex-col gap-4 lg:flex-row lg:items-end lg:justify-between">
7:   <div>
8:     <div class="flex items-center gap-2 text-xs font-bold uppercase tracking-[0.16em] text-indigo-500">
9:       <span>ClubDash</span><span class="text-slate-300">/</span><span>Bengaluru Sports Network</span>
10:     </div>
11:     <h1 class="mt-2 text-3xl font-extrabold tracking-tight text-slate-900">Nearby Courts & Sports Clubs</h1>
12:     <p class="mt-1 max-w-2xl text-sm text-slate-500">Discover sports venues across Bengaluru, preview real turn-by-turn road routes, and reserve your court in seconds.</p>
13:   </div>
14:   <div class="flex flex-wrap items-center gap-2.5">
15:     <button class="btn-secondary" @click="centerOnBangalore">Bengaluru Center</button>
16:     <button class="btn-secondary" @click="fitMap">Fit All</button>
17:     <button class="btn-primary" :disabled="locating" @click="locateMember">
18:       {{ locating ? 'Locating GPS...' : userLocation ? 'Update Location' : 'Use My Location' }}
19:     </button>
20:   </div>
21: </div>

```

Line #	Exact Code Snippet	What It Renders & Does in the Live UI
Line 11	<h1 class="text-3xl font-extrabold">Nearby Courts...</h1>	Renders bold header title "Nearby Courts & Sports Clubs" .
Line 15	<button @click="centerOnBangalore">Bengaluru Center</button>	Re-centers the map to central Bengaluru coordinates (12.9716, 77.5946).

Line 16	<button @click="fitMap">Fit All</button>	Adjusts zoom and pan to fit all sports venues across the city in view.
Lines 17–19	<button @click="locateMember">Use My Location</button>	Triggers GPS detection, computes distance to each club, and displays blue user pin ■.

B. [Box 3] Multi-Filter Bar: Search, Radius & Sports (Lines 150 – 257)

```
150: <div class="glass p-4 sm:p-5 rounded-2xl space-y-4">
151:   <div class="grid gap-3 md:grid-cols-[1fr_200px]">
152:     <input v-model="searchQuery" type="text" placeholder="Search clubs, localities..." @input="onSearchChange" />
153:     <select v-model="selectedDistanceRadius" :disabled="!userLocation" @change="applyFilters">
154:       <option value="all">{{ userLocation ? 'Any Distance' : 'Distance (Enable GPS)' }}</option>
155:       <option value="5">Within 5 km</option>
156:       <option value="10">Within 10 km</option>
157:     </select>
158:   </div>
159:   <div class="flex items-center gap-1.5 overflow-x-auto">
160:     <button v-for="sport in sportOptions" :key="sport.value" class="sport-pill"
161:       :class="{ 'sport-pill-active': selectedSport === sport.value }" @click="selectSport(sport.value)">
162:       <span>{{ sport.emoji }}</span><span>{{ sport.label }}</span>
163:       <span class="sport-pill-count">{{ sportCounts[sport.value] }}</span>
164:     </button>
165:   </div>
166:   <div class="flex justify-between text-xs text-slate-500">
167:     <div>Showing <b>{{ filteredClubs.length }}</b> clubs with <b>{{ totalFilteredCourts }}</b> courts</div>
168:     <button v-if="isFiltered" @click="resetFilters">Reset all filters</button>
169:   </div>
170: </div>
```

Line #	Exact Code Snippet	What It Renders & Does in the Live UI
Line 152	<input v-model="searchQuery" @input="onSearchChange">	Renders keyword search filter matching club name, locality, or sport.
Lines 153–157	<select v-model="selectedDistanceRadius">	Distance filter dropdown (Within 5 km, 10 km, 25 km). Disabled until GPS is enabled.
Lines 160–164	v-for="sport in sportOptions" ... @click="selectSport"	Renders horizontal scrolling sport pills with live badge counts: All Sports (37), Tennis (9), Badminton (9) , etc.
Line 167	Showing {{ filteredClubs.length }} clubs with {{ totalFilteredCourts }} courts	Renders live counter: "Showing 10 clubs with 37 courts" .

C. [Box 5] In-App Road Navigation & Turn-by-Turn GPS (Lines 361 – 495)

```
361: <div v-if="activeRoute" class="glass p-5 rounded-2xl border-2 border-indigo-500 bg-gradient-to-br from-indigo-50/60 to-white shadow-md space-y-3.5">
362:   <div class="flex items-start justify-between">
363:     <div>
364:       <span class="px-2 py-0.5 rounded-md bg-indigo-600 text-[10px] font-extrabold text-white">■■ In-App Road Route</span>
365:       <h3 class="mt-1 text-lg font-extrabold text-slate-900">To {{ activeRoute.destinationClub.name }}</h3>
366:     </div>
367:     <button @click="clearRoute">X</button>
368:   </div>
369:   <!-- Travel Time & Distance -->
370:   <div class="grid grid-cols-2 gap-2.5">
371:     <div class="p-2.5 bg-white rounded-xl text-center">
372:       <p class="text-[10px] text-slate-400">EST. TRAVEL TIME</p>
373:       <p class="text-xl font-black text-indigo-600">{{ activeRoute.durationMin }} mins</p>
374:     </div>
375:     <div class="p-2.5 bg-white rounded-xl text-center">
376:       <p class="text-[10px] text-slate-400">ROAD DISTANCE</p>
377:       <p class="text-xl font-black text-slate-800">{{ activeRoute.distanceKm }} km</p>
378:     </div>
379:   </div>
380:   <!-- Mode Switcher -->
381:   <div class="flex gap-2 bg-slate-100 p-1 rounded-xl">
382:     <button :class="{ 'bg-indigo-600 text-white': routeMode === 'driving' }" @click="changeRouteMode('driving')">■ Driving (Car/Bike)</button>
383:     <button :class="{ 'bg-indigo-600 text-white': routeMode === 'walking' }" @click="changeRouteMode('walking')">■ Walking</button>
384:   </div>
385:   <!-- Turn-by-Turn Collapsible Steps -->
386:   <button @click="showTurnByTurn = !showTurnByTurn">Turn-by-Turn Directions ({{ activeRoute.steps.length }} steps)</button>
387:   <div v-if="showTurnByTurn">
388:     <div v-for="(step, idx) in activeRoute.steps" :key="step.id" class="p-2 bg-white rounded-lg flex gap-2">
389:       <span>{{ idx + 1 }}</span><p>{{ step.instruction }}</p><span>{{ step.distanceText }}</span>
390:     </div>
391:   </div>
392: </div>
```

Line #	Exact Code Snippet	What It Renders & Does in the Live UI
Lines 364–365	To {{ activeRoute.destinationClub.name }}	Renders route target header (e.g. "To The Koramangala Club").
Lines 372–378	{{ activeRoute.durationMin }} mins & {{ activeRoute.distanceKm }} km	Displays bold travel time ("9 mins") and driving distance ("5.4 km").
Lines 381–384	changeRouteMode('driving') vs changeRouteMode('walking')	Toggles between vehicle and pedestrian routing calculations.
Lines 386–391	v-for="step in activeRoute.steps"	Renders step-by-step turn instructions (e.g. "Turn right onto Sarjapur Road").